breedRBayes: ASReml-Style Formula Front-End for BGLR

September 2, 2026

Contents

`as_mcmc` *Convert a fitted model's posterior draws to a* `coda` *object*

Description

Reads the variance-component chains (and optionally the intercept `mu`) written by BGLR and returns them as a coda::mcmc.list (one component per chain), ready for `mcmc_diag()` or `coda`/`bayesplot` functions.

Usage

```
as_mcmc(x, ...)

## S3 method for class 'breedRB_fit'
as_mcmc(x, what = "varcomp", ...)
```

Arguments

x	A breedRB_fit.
...	Unused.
what	Which quantities to return: any of "varcomp" and "mu".

Value

A coda::mcmc.list.

---

bbglr *Fit a Bayesian mixed model with the BGLR engine using asreml-style formulas*

Description

Fit a Bayesian mixed model with the BGLR engine using asreml-style formulas

Usage

```
bbglr(
  fixed,
  random = NULL,
  residual = NULL,
  data,
  relmat = list(),
  nIter = 5000,
  burnIn = 1000,
  thin = 5,
  nChains = 1,
  seed = 123,
  saveAt = NULL,
  verbose = TRUE
)
```

Arguments

fixed	Two-sided formula for the fixed/mean part, e.g. `yield ~ 1 + env`. Use `cbind(t1, t2) ~ ...` for a multi-trait model.
random	One-sided formula of random terms, e.g. `~ vm(gen, G)`. Special functions: `vm(f, K)` genomic effect with relationship matrix `K`; `leg(x, n)` / `rr(x, n)` random regression of order `n`; `fa(f, k)` / `rrc(f, k)` factor-analytic. Bare names are factors; `a:b` is an interaction.
residual	One-sided residual formula. `NULL`/`~ units` = homogeneous; `~ dsum(~units | env)` = heterogeneous residual variances by `env`.
data	A data frame.
relmat	Named list of relationship / covariance matrices referenced by `vm()` (e.g. `list(G = kinship)`). Matrices use covariance (K), not its inverse. Row/column names must match the factor levels.
nIter, burnIn, thin	MCMC controls passed to BGLR.
nChains	Integer number of independent chains (distinct seeds). Enables Gelman-Rubin diagnostics when `> 1`.
seed	Base random seed; chain `c` uses `seed + c - 1`.
saveAt	Directory for BGLR binary output. Defaults to a fresh temp dir.
verbose	Logical; print BGLR progress.

Value

An object of class `breedRB_fit`.

Examples

```
G <- readRDS(system.file("extdata", "kinship.matrix.rds", package = "breedRBayes"))
dat <- read.csv(system.file("extdata", "data_soy.csv", package = "breedRBayes"))
fit <- bbglr(yield ~ 1 + env, random = ~ vm(gen, G), data = dat,
             relmat = list(G = G), nIter = 2000, burnIn = 500, nChains = 2)
```

---

extr_outs_bglr *Extract posterior summaries, Legendre random regression coefficients, variance components, diagnostics, and predictive profiles from FW_bglr*

Description

Summarizes posterior chains, variance components, diagnostic traces, and genotype-level Finlay–Wilkinson random regression parameters (intercept and Legendre coefficients) from a model fitted by `FW_bglr`. Also constructs environment-by-genotype predictions and per-genotype R^2 against observed means.

Usage

```
extr_outs_bglr(model, gen)
```

Arguments

model	An object of class bglr_met returned by FW_bglr, including its path (saveAt), posterior effect matrices (INT, COEF, SLOPE, GE, GGE), and metadata (data, trait, env, order, check, X_gradient, range).
gen	Character. Column name for genotype IDs, matching the column used in FW_bglr.

Details

Conceptual model with Legendre basis and separate intercept: #$# y_ge = \mu + l_e + \alpha_g + \sum_d=1^D \gamma_g,d\,\tilde P_d(x_e) + \varepsilon_ge #$# where #$#x_e#$# is the environment index scaled to -1,1, #$#\tilde P_d#$# are orthonormal Legendre polynomials for degrees #$#d=1,...,D#$#, and #$#\alpha_g#$# is the genotype-specific intercept. The function collects posterior chains for #$#\alpha_g#$# and #$#\gamma_g,d#$#, variance components, and diagnostics.

Files read for diagnostics:

- mu.dat, ETA_int_varB.dat, ETA_l_varB.dat, and ETA_slope{d}_varB.dat (for each degree).
- Ensure FW_bglr was run with saveAt pointing to a writable directory that ends with a path separator (e.g., "/"), so these files exist at extraction time.

Predictions and R^2:

- Environment-by-genotype predictions use the model's environmental basis X_gradient (columns deg0..degD, with deg0=1) and posterior mean coefficients (intercept and slope{d}).
- Observed means by genotype×environment are computed from model$data, and R^2 is the squared correlation between predictions and observed means across environments per genotype.

Value

An object of class extr_bglr (list) containing:

- post: list of posterior chains:
  - chain_int nGen x nIter: genotype intercepts.
  - chain_coef: list of degree-specific chains (slope1..slopeD), each nGen x nIter.
  - chain_slope: first-degree chain for backward compatibility (may be NULL if order==0).
  - chain_ge: centered genotype-by-environment predictions across iterations.
  - chain_gge: uncentered genotype + GE predictions across iterations.
- variances: data frame with posterior means and SDs of variance components: intercept, environment main effect (l), each slope_d, and error (from Step 2).
- diagnostic_trace: data frame with traces for mu, intercept, l, each slope{d}, and iteration index.

- effective_sizes: numeric vector of effective sample sizes for diagnostic traces computed via coda::effectiveSize.
- gradient: matrix with two columns (intercept=1, gradient values) spanning the range of the environment effect (l) for legacy compatibility.
- gradient_basis: Legendre basis matrix (deg0..degD) on the adaptive grid returned by .make_gradient_basis, suitable for evaluating profile curves.
- G_param: data frame with genotype-level posterior means for intercept and slope{d} (degrees 1..D), plus a check flag.
- N_obs: data frame with counts of observations per genotype (ID, Obs).
- check: vector of check genotype IDs propagated from the model.
- ID_R2: data frame with per-genotype R^2 (squared correlation across environments between predicted and observed means).

See Also

FW_bglr, .make_gradient_basis, legendre_basis

Examples

```
# Fit FW_bglr first (see FW_bglr examples), then:
# ext <- extr_outs_bglr(model = fit_fw, gen = "gid")
# names(ext)
# head(ext$G_param)
# ext$variances
# ext$effective_sizes
# str(ext$gradient_basis)
```

FW_bglr *Bayesian Multi-Environment Trial Analysis with BGLR via Random Regression (Finlay–Wilkinson)*

Description

Fits the Finlay–Wilkinson model in two steps using the BGLR package, with genotype effects through a genomic relationship matrix, environment main effects, and a random-regression slope per genotype based on the environment index estimated in Step 1.

Usage

```
FW_bglr(
  data,
  gen,
  env,
  trial = NULL,
```

```
  block = NULL,
  check = NULL,
  trait,
  res.het = FALSE,
  nIter = 2000,
  burnIn = 500,
  verbose = TRUE,
  saveAt = "",
  kinship.matrix = NULL,
  W = NULL,
  order = 1,
  seed = 123
)
```

Arguments

`data`	A data frame containing the multi-environment trial data.
`gen`	Character. Column name for genotype IDs.
`env`	Character. Column name for environment IDs.
`trial`	Character (optional). Column name for experiment/trial factor (if multiple trials within environments).
`block`	Character (optional). Column name for replication/block factor.
`check`	Character (optional). Column name with logical values (TRUE/FALSE) marking check genotypes.
`trait`	Character. Column name for the response variable (phenotype).
`res.het`	Logical. Heterogeneous residuals? (Present for compatibility; not used directly). Default = FALSE.
`nIter`	Integer. Number of MCMC iterations for BGLR. Default = 2000.
`burnIn`	Integer. Burn-in period. Default = 500.
`verbose`	Logical. Print progress from BGLR? Default = TRUE.
`saveAt`	Character. Path/prefix to save binary files from BGLR (used later by `readBinMat`). Default = "" (current working directory).
`kinship.matrix`	Matrix (required). Genomic relationship matrix with row/column names matching genotype IDs in `data[[gen]]`.
`seed`	Integer. Random seed for reproducibility. Default = 123.

Details

Conceptually, the model can be written as: #$# y_ge = \mu + l_e + \alpha_g + \beta_g \, l_e + \varepsilon_ge #$# where y is the phenotype for genotype g in environment e, l_e is the environment index, alpha_g is the genotype-specific intercept, beta_g is the genotype-specific slope (sensitivity to environment), and epsilon is the residual.

Workflow:

- Step 1: Fit genotype (G) and environment (E) main effects via BGLR; obtain the environment projections to build an environmental index.

- Step 2: Fit genotype-specific intercept and slope (G + G×E) using the index derived from Step 1 through a random-regression formulation.

Implementation notes:

- The genomic relationship matrix is aligned internally to the genotype incidence matrix (Zg) order, ensuring consistency between model inputs.
- The environment design matrix is centered to represent the environmental index used in the slope computation.
- Posterior samples for intercept and slope are read from the binary files saved by BGLR (`ETA_int_b.bin`, `ETA_slope_b.bin`), which depend on `saveAt`.
- BLAS threading is controlled via RhpcBLASctl for reproducibility/performance.

Checks performed by the code:

- Verifies that specified columns (`gen`, `env`, `trait`, and optionally `trial`, `block`, `check`) exist in `data`.
- Filters `data` to genotypes present in `rownames(kinship.matrix)`.
- Aligns `kinship.matrix` to match the genotype incidence matrix (`Zg`) ordering/content.

Value

An object of class `bglr_met` containing:

- `path`: the `saveAt` path used to write/read BGLR binaries.
- `GGE`: wide matrix (by genotype:environment rows) with raw predictions (G + GE) across iterations.
- `GE`: wide matrix (by genotype:environment rows) with centered predictions (mu + XB) across iterations.
- `INT`: matrix of genotype intercepts by iterations.
- `SLOPE`: matrix of genotype slopes by iterations.
- `fit1`: BGLR fit object for Step 1 (G + E).
- `fit2`: BGLR fit object for Step 2 (G + GE).
- `data`: curated/aligned data used in the model.
- `trait`, `gen`, `env`, `trial`, `block`: column names used in the analysis.
- `check`: vector of check genotype IDs (if `check` column was provided).

References

Finlay, K. W., & Wilkinson, G. N. (1963). The analysis of adaptation in a plant-breeding programme. Australian Journal of Agricultural Research, 14(6), 742–754.

Examples

```
# Example using EnvRtype data
library(EnvRtype)
data("maizeG")     # genomic relationship matrix
data("maizeYield") # phenotypic data

data <- maizeYield
kinship.matrix <- maizeG

# Select 5 random genotypes as checks (as in your setup)
set.seed(123)
check_ids <- as.character(sample(unique(data$gid), 5))
data$check <- data$gid %in% check_ids

# Identify a candidate trait column (exclude ID-like columns)
candidate_traits <- setdiff(names(data), c("gid","env","trial","block","check"))
trait_col <- candidate_traits[1] # use the first available trait column

# Create a temporary output directory for BGLR binary files
out_dir <- tempfile(pattern = "bglr_fw_")
dir.create(out_dir)

# Run the Finlay-Wilkinson two-step model
fit_fw <- FW_bglr(
  data = data,
  gen = "gid",
  env = "env",
  trait = trait_col,
  check = "check",
  kinship.matrix = kinship.matrix,
  nIter = 2000,
  burnIn = 500,
  saveAt = paste0(out_dir, "/"),
  verbose = TRUE
)

# Inspect results
names(fit_fw)
dim(fit_fw$INT)   # intercepts by genotype x iterations
dim(fit_fw$SLOPE) # slopes by genotype x iterations
dim(fit_fw$GE)    # genotype:environment rows x iterations
dim(fit_fw$GGE)   # same, uncentered

# Check genotypes
fit_fw$check
```

---

gebv *Posterior genomic estimated breeding values (GEBV)*

Description

Reconstructs per-level genetic values for a `vm()` term from the saved BGLR effect samples, mapping the ridge coefficients back through the PC rotation (`genetic value = PC %*% b`), pooled across chains.

Usage

```
gebv(fit, term = NULL, prob = 0.95)
```

Arguments

`fit`	A `breedRB_fit` (single-trait).
`term`	Genetic term identifier (label or key). Defaults to the first `vm()` term.
`prob`	Central credible-interval mass (default 0.95).

Value

A data frame with `ID`, posterior `gebv` (mean), `sd`, `lower`, `upper`, ordered by decreasing `gebv`, with the pooled draws matrix (`nDraws x nLevel`) attached as attribute `"draws"`.

Examples

```
head(gebv(fit))
```

`heritability` *Posterior distribution of heritability*

Description

Computes narrow-sense (genomic) heritability for every MCMC draw, returning the full posterior distribution rather than a single point estimate:

$$h^2 = \sigma^2_{genetic} / (\sigma^2_{genetic} + \sum \sigma^2_{other} + \sigma^2_e).$$

Usage

```
heritability(fit, genetic = NULL, denominator = NULL, prob = 0.95)
```

Arguments

`fit`	A `breedRB_fit`.
`genetic`	Character vector of genetic term identifiers (label or key) forming the numerator. Defaults to the `vm()` term(s).
`denominator`	Character vector of term identifiers summed in the denominator alongside `varE`. Defaults to all random terms.
`prob`	Central credible-interval mass (default 0.95).

Value

A list of class `breedRB_h2`: `summary` (mean/median/sd/CI) and `draws` (posterior vector of h^2).

Examples

```
h2 <- heritability(fit); h2$summary; hist(h2$draws)
```

`legendre_basis` *Orthogonal / orthonormal Legendre polynomial basis on [-1, 1]*

Description

Constructs a Legendre polynomial basis evaluated at a numeric vector scaled to [-1, 1], up to a specified order, using the three-term recurrence. Optionally returns the orthonormal basis.

Usage

```
legendre_basis(x, order, orthonormal = FALSE)
```

Arguments

`x`	Numeric vector (already scaled to [-1, 1]) where the basis is evaluated.
`order`	Integer. Maximum polynomial degree. Returns columns for degrees 0..order.
`orthonormal`	Logical. If `TRUE`, applies orthonormal scaling to each degree.

Value

A numeric matrix `length(x) x (order + 1)`; column `j` is the degree `j - 1` polynomial.

Examples

```
legendre_basis(seq(-1, 1, length.out = 5), order = 3)
```

`mcmc_diag` *Markov-chain convergence diagnostics for a fitted model*

Description

Summarises convergence of the variance-component chains: effective sample size, Geweke's z, and (when the fit has >1 chain) the Gelman-Rubin potential-scale-reduction factor (R-hat).

Usage

```
mcmc_diag(fit, what = "varcomp")
```

Arguments

fit	A breedRB_fit.
what	Passed to as_mcmc() (default "varcomp"; add "mu" to include the intercept).

Value

A data frame with one row per monitored parameter: param, n_eff, geweke_z, and Rhat (NA for single-chain fits).

Examples

```
mcmc_diag(fit)
```

---

parse_model *Parse asreml-style fixed / random / residual formulas*

Description

Turns the three model formulas into an internal, engine-agnostic description used by bbglr(). Not usually called directly.

Usage

```
parse_model(fixed, random = NULL, residual = NULL)
```

Arguments

fixed	Two-sided formula for the mean/fixed part, e.g. yield ~ 1 + env. Multi-trait models use cbind(t1, t2) ~ ....
random	One-sided formula for random terms, e.g. ~ vm(gen, G) + env:vm(gen, G). NULL for none.
residual	One-sided residual formula. NULL (default) or ~ units gives homogeneous residuals; ~ dsum(~units | env) gives heterogeneous residual variances by env.

Value

A breedRB_terms list: response, fixed, random, residual.

Examples

```
parse_model(yield ~ 1 + env, ~ vm(gen, G))
```

`plot_fw` *Plot Finlay–Wilkinson regression lines highlighting Top-N and checks*

Description

Draws genotype-specific regression profiles across the environment gradient, highlighting the Top-N genotypes (by mean predicted value across the gradient) and the check genotypes.

Usage

```
plot_fw(post_prob, top_n = 10)
```

Arguments

`post_prob`	An object produced by `prob_sup_bglr()`. It must contain `post_prob$across$reg`, a data frame with columns: • `gradient`: environmental index values, • `value`: predicted response along the gradient, • `ID`: genotype ID, • `check`: logical flag indicating checks.
`top_n`	Integer. Number of top genotypes to highlight (default = 10).

Details

Input compatibility:

- Uses `post_prob$across$reg` created from the Legendre basis (`gradient_basis`) in `prob_sup_bglr`, where predictions across the adaptive gradient are computed as #$# \haty*g(l)* = *sum*d=0^D \beta_g,d\,\mathrmdeg_d(l) #$#, with `deg0` = 1 and `deg1..degD` being orthonormalized Legendre basis columns.
- The Top-N genotypes are defined by the highest mean of `value` across the gradient.
- Lines are colored and thickened for `TopN` and `Check` categories; others are thin and grey.
- Labels are added at the last gradient point only for highlighted lines.
- The legend labels adapt dynamically to `top_n` (e.g., "Top10", "Top30").

Value

A ggplot object.

See Also

prob_sup_bglr

Examples

```
# Assuming you already ran prob_sup_bglr():
p1 <- plot_fw(post_prob = probs, top_n = 10)
print(p1)
```

`plot_pair_prob` *Heatmap of pairwise probabilities of superior performance (Top-N)*

Description

Visualizes pairwise probabilities among the Top-N genotypes using the matrix of pairwise performance probabilities computed in `prob_sup_bglr()`.

Usage

```
plot_pair_prob(post_prob, top_n = 30)
```

Arguments

`post_prob`	Output from `prob_sup_bglr()` containing: • `across$bayesian_stats` (for ranking by `prob_top`), • `across$pair_perfo` (square matrix of pairwise probabilities), • `control$increase` (flag indicating whether higher is better), typically set in `prob_sup_bglr`.
`top_n`	Integer. Number of top genotypes to display. Default = 30.

Details

- The function subsets the pairwise matrix to the Top-N genotypes and masks the lower triangle and diagonal for clearer visualization.
- The legend label is determined by the `increase` flag: `"Pr(X > Y)"` if higher is better, otherwise `"Pr(X < Y)"`.

Value

A ggplot object.

See Also

prob_sup_bglr

Examples

```
p6 <- plot_pair_prob(post_prob = probs, top_n = 30)
print(p6)
```

plot_param *Scatter of FW parameters: slope vs intercept colored by R2*

Description

Visualizes genotype-level FW parameters with slope on the x-axis and intercept on the y-axis, coloring by an R2 statistic (e.g., goodness-of-fit of the FW regression per genotype).

Usage

```
plot_param(post_prob)
```

Arguments

post_prob	Output from prob_sup_bglr() containing across$bayesian_stats with columns slope, intercept, and an R2 column.

Details

Legendre-compatible:

- The slope here corresponds to the degree-1 coefficient from the Legendre random regression (as reported in bayesian_stats$slope). Intercept is #$# \alpha_g #$#.
- This function assumes that post_prob$across$bayesian_stats contains an R2 column. If R2 was not computed earlier, you should add it before calling this function (e.g., based on observed vs fitted FW lines per genotype).

Value

A ggplot object.

See Also

prob_sup_bglr

Examples

```
# If bayesian_stats has an R2 column:
p4 <- plot_param(post_prob = probs)
print(p4)
```

plot_posterior *Posterior density plot of variance components or heritability*

Description

Posterior density plot of variance components or heritability

Usage

```
plot_posterior(x, ...)

## S3 method for class 'breedRB_fit'
plot_posterior(x, ...)

## S3 method for class 'breedRB_h2'
plot_posterior(x, ...)
```

Arguments

x	A breedRB_fit (variance components) or a breedRB_h2 object.
...	Unused.

Value

A ggplot object.

plot_prob_intercept *Lollipop plot of across-environment top-tail probability (intercept-based)*

Description

Plots the probability of being in the top tail (as defined in prob_sup_bglr, e.g., top 20%) for genotype intercepts, highlighting Top-N genotypes and checks.

Usage

```
plot_prob_intercept(post_prob, top_n = 30)
```

Arguments

post_prob	Output from prob_sup_bglr() containing across$bayesian_stats with columns ID, check, intercept, and prob_top.
top_n	Integer. Number of genotypes to highlight by highest average intercept (default = 30).

Details

Legendre compatibility:

- Intercepts are genotype-level #$# \alpha_g #$# means; slope terms (Legendre coefficients) do not enter here. Top-N is ranked by the mean intercept.
- The Top-N set is formed by ranking genotypes on the mean `intercept`.
- The y-axis shows `prob_top`: probability of being in the global top tail defined by `int` (from `prob_sup_bglr(control$int)`).

Value

A ggplot object.

See Also

prob_sup_bglr

Examples

```
p2 <- plot_prob_intercept(post_prob = probs, top_n = 30)
print(p2)
```

`plot_prob_slope` *Lollipop plot of slope sensitivity probability (Pr(beta > 1))*

Description

Plots the probability of degree-1 slope being greater than 1 for Top-N genotypes and checks, highlighting sensitivity to the environment index relative to 1.

Usage

```
plot_prob_slope(post_prob, top_n = 30)
```

Arguments

`post_prob`	Output from `prob_sup_bglr()` containing `across$bayesian_stats` with columns `ID`, `check`, and `slope_gt_1`.
`top_n`	Integer. Number of genotypes to highlight by highest average slope (default = 30).

Details

Legendre note:

- Under orthonormal Legendre basis, the degree-1 term multiplies #$# \tilde P_1(x) #$#, so a threshold at 0 is more natural for directionality; `> 1` is kept for backward compatibility.
- Top-N set is formed by ranking genotypes on the mean `slope` (degree-1 coefficient).

Value

A ggplot object.

See Also

prob_sup_bglr

Examples

```
p3 <- plot_prob_slope(post_prob = probs, top_n = 30)
print(p3)
```

---

plot_prob_within *Heatmap of within-environment top-tail probabilities*

Description

Draws a heatmap of environment-specific probabilities for Top-N genotypes, where the probabilities represent within-environment tail probabilities computed in `prob_sup_bglr()`.

Usage

```
plot_prob_within(post_prob, top_n = 30)
```

Arguments

`post_prob`	Output from `prob_sup_bglr()` containing: • `across$bayesian_stats` with `ID` and `prob_top` for ranking, • `within$perfo` with columns `env`, `ID`, and `prob` for environment-specific probabilities.
`top_n`	Integer. Number of genotypes to display (ranked by `prob_top`). Default = 30.

Details

Legendre-compatible:

- Probabilities are based on environment-specific posterior predictions from the Legendre random regression (`within$perfo`).

Value

A ggplot object.

See Also

prob_sup_bglr

Examples

```
p5 <- plot_prob_within(post_prob = probs, top_n = 30)
print(p5)
```

`plot_trace` *Trace plots of the variance-component chains*

Description

Trace plots of the variance-component chains

Usage

```
plot_trace(fit, what = "varcomp")
```

Arguments

`fit`	A `breedRB_fit`.
`what`	Passed to `as_mcmc()`.

Value

A `ggplot` object (iteration on x, value on y, one panel per parameter, coloured by chain).

`predict_env_gradient_pca` *Predict Environment Gradient (Index) for New Environments via PCA Projection*

Description

Uses the PCA mapping and coefficients learned in `FW_bglr` (when environmental covariates `W` were provided) to predict the environment gradient/index #$# l_e #$# for a new set of environments.

Usage

```
predict_env_gradient_pca(model, Wp)
```

Arguments

model	A fitted object from FW_bglr with pca and beta_w components populated (i.e., W was provided during training).
Wp	Numeric matrix of environmental covariates for new environments (rows = environments, columns = same covariates used in training). Row names should be the environment IDs.

Details

Procedure:

1. Standardize the new Wp covariate matrix with the original training means and sds.
2. Project standardized Wp onto the retained PCA loadings (scores).
3. Standardize the new scores using the training score center/scale.
4. Multiply by the learned regression coefficients #$# \beta #$# from Step 1 to obtain #$# l_e #$#.

Value

A named numeric vector of predicted environment gradient values #$# l_e #$#, with names taken from rownames(Wp).

Examples

```
# Suppose fit_fw was obtained with W provided:
# new_W is a matrix with the same columns as fit_fw$W and rownames as environment IDs
# l_new <- predict_env_gradient_pca(fit_fw, new_W)
```

---

prob_best_by_gradient *Probability that each genotype is the winner along an environment gradient*

Description

For each point of a (predicted) environment gradient, computes the posterior probability that each genotype has the maximum predicted value, from the random-regression posterior draws of a FW_bglr() fit.

Usage

```
prob_best_by_gradient(model, l_new, tie.method = c("share", "first"))
```

Arguments

model	An object returned by FW_bglr().
l_new	Named numeric vector of environment-index values to evaluate.
tie.method	"share" splits the count equally among ties; "first" assigns the win to the first tied genotype.

Value

A list with prob (genotype x gradient probability matrix) and winner (most probable genotype per gradient point).

See Also

FW_bglr(), predict_env_gradient_pca()

---

prob_sup_bglr *Posterior probability summaries and rankings from extr_outs_bglr output*

Description

Computes posterior-based probabilities and summaries for genotype performance, stability, environment-specific performance, and Legendre random regression coefficients, using the output produced by extr_outs_bglr (which itself summarizes a FW_bglr fit).

Usage

```
prob_sup_bglr(extr, int = 0.2, increase = TRUE, verbose = FALSE)
```

Arguments

extr	An object of class extr_bglr created by extr_outs_bglr, containing posterior chains, observation counts, gradient matrices (gradient, gradient_basis), genotype parameters, and check IDs.
int	Numeric in (0,1). Intensity parameter defining the tail probability used for "top/bottom" performance thresholds (e.g., 0.2 means top 20%). Default = 0.2.
increase	Logical. If TRUE, higher values of the response are better (e.g., yield); if FALSE, lower values are better (e.g., disease scores). Default = TRUE.
verbose	Logical. Currently unused; reserved for future messages. Default = FALSE.

Details

Under the Finlay–Wilkinson formulation with Legendre basis and separate intercept: #$# y_ge = \mu + l_e + \alpha_g + \sum_d=1^D \gamma_g,d\,\tilde P_d(x_e) + \varepsilon_ge #$# this function provides probabilities and summaries for #$# \alpha_g #$# (genotypic intercepts), the degree-specific Legendre coefficients #$# \gamma_g,d #$#, and their environment-specific predictions.

Specifically, it computes:

- Across-environment performance probability: fraction of posterior samples in which a genotype's intercept ranks in the top tail defined by `int` (or bottom tail if `increase = FALSE`).
- Pairwise performance probability: #$# P\left(\alpha_i > \alpha_j\right) #$# (or #$# < #$# if `increase = FALSE`).
- Across-environment stability probability: fraction of posterior samples in which the variance of the genotype-specific centered G×E predictions is below the `int`-quantile (i.e., more stable).
- Pairwise stability probability: #$# P\left(\mathrmVar_e(\textGE*i)* < *mathrmVar*e(\textGE_j)\right) #$#.
- Joint probability: product of performance and stability probabilities (heuristic).
- Within-environment performance probability: fraction of posterior samples for each environment in which the genotype's environment-specific prediction ranks in the top tail defined by `int` (or bottom tail if `increase = FALSE`).
- Posterior summaries (medians, SDs, 2.5% and 97.5% quantiles) for intercepts and all Legendre coefficients across degrees (returned in `hpd_coef`).
- Probability of being better than the best check (based on posterior draws).
- An approximate t-test-style comparison versus the best check using posterior SDs and observed counts.

Note: Probabilities related to slope being greater than 1 (or less than 1) are not computed in this version.

Inputs expected from `extr_outs_bglr`:

- `post$chain_int` (matrix): posterior samples of genotype intercepts (# genotypes × iterations).
- `post$chain_coef` (list): posterior samples per Legendre degree d = 1..D (each # genotypes × iterations).
- `post$chain_ge` (matrix): centered genotype-by-environment posterior predictions (combos × iterations).
- `post$chain_gge` (matrix): uncentered genotype-by-environment posterior predictions (combos × iterations).
- `gradient` (matrix): two columns 1, l, with an intercept of 1 and an environment index l (legacy).
- `gradient_basis` (matrix): Legendre basis on an adaptive grid (columns `deg0..degD`, with `deg0=1`).
- `G_param` (data frame): genotype-level posterior means for intercept and slopes `slope{d}`, plus a `check` flag.
- `N_obs` (data frame): counts of observations per genotype.

- check (character): vector of check genotype IDs.

Interpretation notes:

- Performance probabilities are computed using global posterior thresholds (across all genotypes and samples), via empirical quantiles defined by int.
- Stability probabilities use the posterior distribution of the variance across environments for each genotype (computed per sample), comparing to the lower tail defined by int.
- The "best check" is defined by the highest posterior median among check genotypes; probabilities of being better than the best check are computed as the fraction of posterior samples in which each genotype's intercept exceeds the best-check intercept sample.
- The t-test-style comparison uses posterior SDs (as SE proxies) and observed counts to construct a simple statistic; treat this as heuristic rather than a strict frequentist test.

Value

An object of class probsup_bglr (a list) with:

- across:
  - g_hpd: posterior summaries for genotype intercepts (medians, quantiles, SD, probability vs best check, t-statistic, p-value, check flag, N).
  - perfo: across-environment performance probabilities (top tail by int or bottom if increase = FALSE).
  - pair_perfo: matrix of pairwise performance probabilities (#$# P(\alpha_i > \alpha_j) #$# or #$# < #$#).
  - stabi: across-environment stability probabilities (low-variance tail).
  - pair_stabi: matrix of pairwise stability probabilities (#$# P(\mathrmVar_e(\textGE_i) < \mathrmVar_e(\textGE_j)) #$#).
  - best_probs: probabilities of being better than the best check.
  - joint_prob: product of performance and stability probabilities.
  - reg: data frame with regression profiles over the environment gradient for each genotype (using gradient_basis).
  - bayesian_stats: consolidated table of main Bayesian summaries (intercept, degree-1 slope, top-prob, best-check-prob).
  - hpd_coef: long-format table with posterior summaries for all degrees (slope1..slopeD).
- within:
  - perfo: environment-specific performance probabilities for each genotype.
  - hpd_gge: posterior summaries for G×E predictions per environment-genotype combo.
  - hpd_slope: posterior summaries for degree-1 slopes (median, SD, 2.5% and 97.5% quantiles) for backward compatibility.

See Also

FW_bglr, extr_outs_bglr

Examples

```
# Assuming you've run FW_bglr and extracted with extr_outs_bglr:
# probs <- prob_sup_bglr(extr = ext, int = 0.2, increase = TRUE)
# names(probs)
# head(probs$across$g_hpd)
# probs$across$perfo[1:5, ]
# head(probs$across$hpd_coef)
```

`prod_summary_by_gradient` *Posterior mean and SD of genotype performance along an environment gradient*

Description

Posterior mean and SD of genotype performance along an environment gradient

Usage

```
prod_summary_by_gradient(model, l_new)
```

Arguments

`model`	An object returned by `FW_bglr()`.
`l_new`	Named numeric vector of environment-index values (e.g. output of `predict_env_gradient_pca()`).

Value

A list with `mean` and `sd` matrices (`genotype x gradient`).

See Also

`FW_bglr()`, `prob_best_by_gradient()`

varcomp *Posterior variance components of a fitted model*

Description

Returns the posterior distribution and summaries of every random-term variance and the residual variance, pooling all chains.

Usage

```
varcomp(fit, prob = 0.95, draws = FALSE)
```

Arguments

fit	A breedRB_fit.
prob	Central credible-interval mass (default 0.95).
draws	Logical; if TRUE also return the pooled posterior draws matrix.

Value

A data frame of per-component summaries (term, mean, median, sd, lower, upper), with the pooled draws attached as attribute "draws" when draws = TRUE.

Examples

```
varcomp(fit)
```